

Relazione Homework 1 Sistemi Distribuiti e Big Data

OpenSky Flight Monitor

Giovanni Maria Contarino 1000007029

Alessia Provvidenza Tomarchio 1000005160

Sommario

1. Introduzione	5
2. Architettura del Sistema	5
2.1 Diagramma Architetturale.....	6
2.2 Descrizione dei Componenti.....	8
3. Scelte Progettuali	9
3.1 Politica At-Most-Once	9
3.2 Persistenza dei dati	10
3.3 Comunicazione Ibrida: REST e gRPC	11
3.4 Sicurezza e Gestione Credenziali.....	11
3.5 Gestione Centralizzata degli Errori	12
4. Variazioni dello Schema Architetturale.....	13
4.1 Introduzione di una Cache Distribuita (Redis).....	13
4.2 Disaccoppiamento tramite Message Broker (Apache Kafka).....	14
4.3 API Gateway (Nginx).....	14
5. Descrizione delle API Implementate	15
5.1 Microservizio: User Manager	15
5.2 Microservizio: Data Collector	16
6. Dettaglio delle Interazioni	17
6.1 Gestione Utenti (Scenari 1 e 2).....	19

6.2 Aggiunta Interessi e Comunicazione Inter-Servizio (Scenario 3)	19
6.3 Ciclo di Vita dello Scheduler (Loop)	20
6.4 Interrogazione Dati e Statistiche (Scenari 5, 6, 7)	20
7. Esempi di Output e Validazione	21
7.1 Verifica <i>At-Most-Once</i> (Cache Hit)	21
7.2 Recupero Dati di Volo (OpenSky)	22
7.3 Statistiche Aggregata (Media Giornaliera)	23
7.4 Recupero Ultimo Volo	24

1. Introduzione

L'obiettivo di questo documento è illustrare le scelte progettuali, tecnologie adottate e i dettagli implementativi del sistema distribuito denominato da noi "Flight Monitor". Lo scopo del sistema è di fornire una piattaforma che permetta agli utenti di registrarsi e monitorare, in *real-time*, il traffico aereo relativo agli aeroporti d'interesse inseriti dagli utenti stessi. L'elaborazione dei dati di volo reali provengono direttamente dalla piattaforma esterna *OpenSky* (<https://opensky-network.org/api>) mediante richieste REST API.

Il sistema è stato realizzato a partire da uno schema architetturale a **microservizi**, containerizzati mediante la piattaforma **Docker** e orchestrati tramite il **Docker Compose**. Questa scelta ha permesso l'isolamento dei microservizi, la loro portabilità su diverse piattaforme e la loro scalabilità. Particolare attenzione è stata posta alla robustezza del sistema, sicurezza dei dati, e efficienza nella comunicazione, così come da specifiche; in particolar modo è stata implementata una gestione centralizzata degli errori, politica *At-Most-Once*, hash dei dati sensibili dell'utente e comunicazione REST e gRPC.

2. Architettura del Sistema

Il sistema è stato decomposto in delle unità funzionali indipendenti, ognuna delle quali risulta responsabile di un singolo dominio applicativo. L'architettura prevede quattro componenti principali (container) che operano all'interno di una rete virtuale gestita da Docker:

1. **User Manager:** Microservizio responsabile della gestione delle identità, dell'autenticazione e della memorizzazione dei dati sensibili degli utenti.

2. **Data Collector:** Microservizio responsabile dell'interazione con le API esterne (OpenSky), della schedulazione dei job di aggiornamento e della fornitura di statistiche sui voli.
3. **Database Relazionale (PostgreSQL):** Supporto di persistenza per lo *User Manager*. Scelto per garantire l'integrità referenziale e la struttura rigida dei dati anagrafici e bancari.
4. **Database NoSQL (MongoDB):** Supporto di persistenza per il *Data Collector*. Scelto per la sua flessibilità nella gestione di documenti JSON complessi e variabili, tipici dei dati di volo grezzi.

In aggiunta ai container, è stata progettata una libreria condivisa (**Shared Library**), che centralizza la logica condivisa tra containers (gestione errori, configurazioni, connettori database).

2.1 Diagramma Architettuale

Di seguito viene riportato il diagramma che illustra i componenti del sistema e le tecnologie di comunicazione adottate.

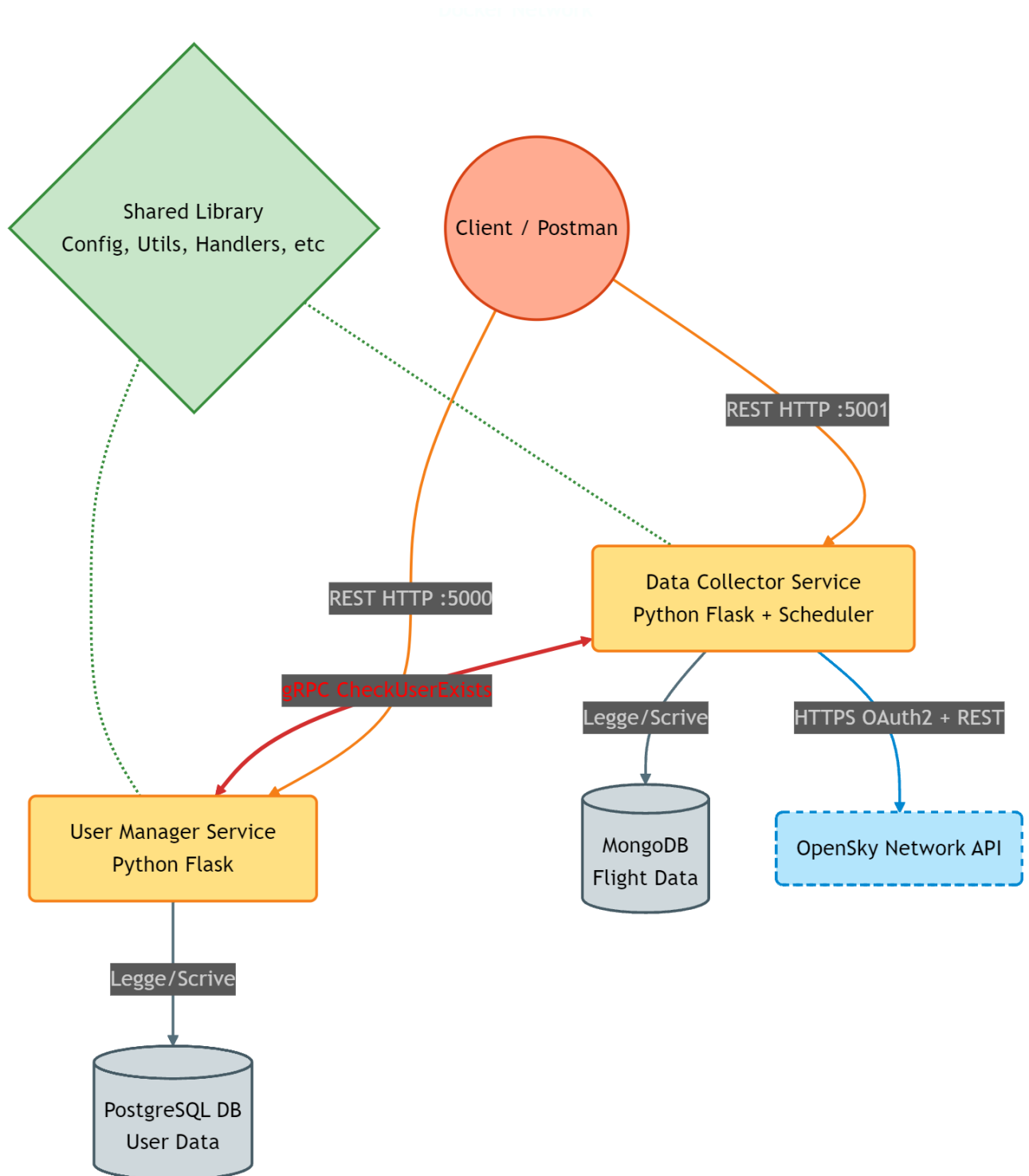


Figura 1. Diagramma architetturale che mostra tutti i micro-servizi coinvolti all'interno del nostro sistema.

2.2 Descrizione dei Componenti

- **User Manager (UM):** consente la registrazione e la cancellazione degli utenti. L'utente attraverso l'utilizzo di un client (nel nostro caso abbiamo sfruttato un plugin implementato di default all'interno dell'IDE IntelliJ IDEA Ultimate, sviluppato da JetBrains, HTTP Client Plugin) che effettua le richieste REST API. Implementa logiche di sicurezza avanzate (hashing per password e dati bancari) e gestisce le richieste di registrazione tramite una cache in memoria (politica *At-Most-Once*). Espone inoltre un'interfaccia gRPC interna per permettere ad altri servizi di verificare l'esistenza di un utente senza accedere direttamente al database.
- **Data Collector (DC):** Agisce come ponte verso il mondo esterno. Implementa un *Job Scheduler* che ciclicamente interroga le API di OpenSky per recuperare voli in arrivo e partenza dagli aeroporti monitorati. Gestisce l'autenticazione verso OpenSky tramite protocollo OAuth2 (le API di OpenSky sono utilizzabili solo tramite il possesso del token OAuth2, poiché l'autenticazione di base è stata deprecata a partire dal mese di marzo 2025).
- **Gestione Dati:** La scelta di utilizzare due tecnologie di database differenti (*PostgreSQL* e *MongoDB*) risponde all'esigenza di ottimizzare la gestione dei dati in base alla loro natura: strutturata e relazionale per gli utenti, non strutturata e ad alta frequenza di scrittura per i dati dei voli.

3. Scelte Progettuali

La progettazione del sistema *Flight Monitor* è stata guidata dalla necessità di soddisfare requisiti quali robustezza, scalabilità e coerenza dei dati. Di seguito vengono analizzate le principali decisioni architetturali adottate.

3.1 Politica At-Most-Once

Uno dei requisiti critici del sistema è garantire che la registrazione di un utente avvenga **al massimo una volta**, anche in presenza di errori di rete che potrebbero causare la ritrasmissione della stessa richiesta (*retry*) da parte del client. Un semplice controllo sull'univocità dell'email a livello di database non è sufficiente a distinguere un *retry* legittimo (stessa richiesta inviata due volte) da un *conflitto* reale (due utenti diversi che vogliono la stessa email).

È stato quindi implementato un meccanismo di gestione dei RequestID tramite l'utilizzo della cache:

1. Il client genera un identificativo univoco per ogni intenzione di scrittura e lo invia nell'header HTTP RequestID.
2. Il servizio *User Manager* mantiene una **Cache in memoria** delle richieste processate con successo.
3. Prima di interrogare il database, il sistema verifica la cache:
 - **Cache Hit:** Se l'ID è presente, il sistema restituisce immediatamente l'esito positivo senza rieseguire la scrittura.

- **Cache Miss:** Se l'ID è nuovo, il sistema procede con la transazione sul database.
4. Per ottimizzare l'uso della memoria, è stata implementata una strategia di pulizia automatica della cache associata ad un *Time-To-Live* (TTL): la cache verifica in automatico la validità temporale del dato (ai fini di correttezza progettuale abbiamo deciso di impostare un TTL pari a 10 minuti), eliminando tutte le richieste che escono fuori dalla finestra temporale di validità.

3.2 Persistenza dei dati

Il sistema abbandona il paradigma del database monolitico in favore della persistenza dei dati, utilizzando tecnologie di storage diverse in base alla natura del dato:

- **PostgreSQL (Database Relazionale alternativo a MySQL):** Utilizzato per lo *User Manager*. I dati utente (anagrafica, credenziali, dati bancari) sono intrinsecamente strutturati e richiedono il rispetto rigoroso dei vincoli di integrità.
- **MongoDB (Database non Relazionale NoSQL):** Utilizzato per il *Data Collector*. I dati provenienti da *OpenSkyNetwork* sono oggetti JSON complessi e semi-strutturati. La scelta di utilizzare MongoDB come database per la gestione dei voli risiede proprio nel modello dei dati, essendo questi restituiti direttamente all'interno di un file JSON, MongoDB li elabora esattamente così come sono tramite un'elaborazione binaria di JSON (BSON), evitando traduzioni complesse o modifiche alle tabelle, nel caso in cui i dati JSON evolvano, gestendo in modo totalmente scalare le enormi quantità di dati che vengono elaborate ad ogni attivazione dello scheduler.

3.3 Comunicazione Ibrida: REST e gRPC

L'interazione tra i componenti sfrutta due protocolli distinti:

- **REST:** Utilizzato per l'interfaccia verso il mondo esterno. È lo standard per le API pubbliche, garantendo interoperabilità e facilità di testing.
- **gRPC:** Utilizzato per la comunicazione interna tra *Data Collector* e *User Manager*.
Quando il Data Collector deve verificare l'esistenza di un utente prima di salvare un interesse, utilizza una chiamata RPC (`CheckUserExists`). Rispetto a REST, gRPC offre minore latenza, serializzazione binaria efficiente e un contratto di interfaccia rigoroso definito tramite file `.proto`.

3.4 Sicurezza e Gestione Credenziali

- **Hashing dei Dati Sensibili:** Le password e i dati bancari non vengono mai salvati in chiaro. Prima di essere salvati nel database, vengono trasformati in hash irreversibili tramite algoritmo **SHA-256**.
- **Gestione Configurazioni:** Per garantire una gestione centralizzata e più flessibile del sistema, nel caso in cui la sua configurazione dovesse cambiare, è stata implementata una classe di configurazione centralizzata (`shared/config.py`) che recupera le variabili d'ambiente essenziali per il funzionamento di tutto il sistema.
- **OAuth2 Client Credentials:** L'integrazione con OpenSky non permette l'utilizzo di un'autenticazione base; pertanto, è stato necessario implementare un flusso di

autenticazione tramite la richiesta ciclica di un *Token* ad ogni esecuzione dello scheduler.

3.5 Gestione Centralizzata degli Errori

In un'architettura a microservizi, se ogni servizio implementasse la gestione delle eccezioni in modo autonomo, il client rischia di ricevere formati di risposta eterogenei (es. HTML, JSON).

Per risolvere questo problema, è stato implementato un modulo condiviso `shared/handlers.py` che definisce gestori universali per le eccezioni HTTP standard:

- **400 Bad Request:** Per dati mancanti o malformati (email o codice aeroporto assente).
- **401 Unauthorized:** Per credenziali errate (password o token).
- **404 Not Found:** Per risorse non trovate (es. volo non trovato).
- **409 Conflict:** Per violazioni di unicità (es. email già in uso o conflitto di RequestID).
- **500 Internal Server Error:** Per eccezioni non gestite.

Ogni microservizio, all'avvio, registra questi handler. In questo modo, qualsiasi eccezione sollevata viene intercettata e trasformata automaticamente in una risposta JSON standardizzata con la struttura: `{"error": "Tipo Errore", "details": "Messaggio specifico"}`.

4. Variazioni dello Schema Architetture

L'architettura attuale è stata progettata per soddisfare i requisiti funzionali dell'homework mantenendo il sistema leggero. Tuttavia, alla luce degli argomenti trattati durante il corso (Redis, Kafka, Nginx), è possibile individuare evoluzioni architetturali orientate a scenari più complessi ed elaborati.

4.1 Introduzione di una Cache Distribuita (Redis)

Attualmente, la gestione della politica *At-Most-Once* è affidata a una cache in-memory all'interno del container *User Manager*.

- **Analisi:** Sebbene funzionale per una singola istanza, questa soluzione non permette la scalabilità orizzontale (repliche multiple dello User Manager non condividerebbero la cache).
- **Evoluzione:** L'introduzione di **Redis** come sistema di cache distribuita permetterebbe di disaccoppiare lo stato della cache dal ciclo di vita dell'applicazione. Redis offrirebbe persistenza, TTL nativo e condivisione dei dati tra multiple repliche del servizio, garantendo la politica *At-Most-Once* anche in un cluster distribuito.

4.2 Disaccoppiamento tramite Message Broker (Apache Kafka)

Nel design attuale, il *Data Collector* comunica in modo sincrono con lo *User Manager* tramite **gRPC** per verificare l'esistenza degli utenti.

- **Analisi:** Sebbene gRPC sia efficiente, crea un accoppiamento temporale: se lo *User Manager* è offline, il *Data Collector* non può salvare nuovi interessi.
- **Evoluzione:** Avendo approfondito a lungo **Apache Kafka** durante il corso, una variazione significativa consisterebbe nel passare a un'architettura **Event-Driven**.
 1. Il *Data Collector* pubblicherebbe un evento su un topic Kafka.
 2. Lo *User Manager* (Consumer) leggerebbe l'evento, validerebbe l'utente e pubblicherebbe l'esito su un topic.
 3. Questa soluzione garantirebbe un disaccoppiamento totale e una maggiore resilienza (i messaggi verrebbero accodati in caso di downtime), sebbene introduca una complessità (gestione del broker, consistenza eventuale).

4.3 API Gateway (Nginx)

Attualmente, i microservizi espongono le loro porte (5000 e 5001) direttamente all'host tramite Docker.

- **Analisi:** In un ambiente reale, esporre direttamente i servizi backend è una pratica insicura e poco flessibile.
- **Evoluzione:** L'introduzione di **Nginx** come *Reverse Proxye API Gateway* permetterebbe di avere un unico punto di ingresso (es. porta 80). Nginx si occuperebbe del routing delle richieste (/api/users verso User Manager, /api/flights verso Data Collector), migliorando la sicurezza e la manutenibilità dell'infrastruttura.

5. Descrizione delle API Implementate

Di seguito sono riportati gli endpoint esposti per le richieste REST da parte del client, suddivise in base al microservizio a cui fanno riferimento. Come già accennato in precedenza, non abbiamo fatto uso di PostMan

5.1 Microservizio: User Manager

- **POST /users** Registrazione di un nuovo utente. La richiesta deve includere l'header RequestID per garantire la proprietà dell'*At-Most-Once*. Il body accetta dati anagrafici (Nome, Cognome, Codice Fiscale) e dati bancari (Carta, Scadenza, CVV). Questi ultimi, insieme alla password, vengono salvati nel database solo sotto forma di hash per garantire la sicurezza.

- **DELETE /users** Cancellazione di un utente. Può essere vista come una Safe-Delete, perché richiede l'invio della password attuale nel corpo della richiesta per autorizzare l'operazione. In caso di successo, l'utente viene rimosso dal database e la cache di idempotenza viene invalidata.

5.2 Microservizio: Data Collector

- **POST /interests** Aggiunge un aeroporto alla lista di monitoraggio di un utente. Prima di salvare il dato su MongoDB, il servizio verifica l'esistenza dell'utente contattando lo *User Manager* tramite protocollo gRPC.
- **GET /flights/{code}** Restituisce la lista grezza degli ultimi mille voli salvati nel database per l'aeroporto specificato (es. EGLL, per Londra), includendo sia gli arrivi che le partenze. Le richieste dell'aeroporto specificato necessitano del codice **ICAO** per funzionare (International Civil Aviation Organization).
- **GET /flights/last** Restituisce l'oggetto JSON relativo all'ultimo volo registrato cronologicamente per un dato aeroporto. Accetta il parametro *type* per specificare se cercare l'ultimo volo in arrivo e/o partenza.
- **GET /statistics/average** Calcola e restituisce la media giornaliera dei voli registrati per un aeroporto negli ultimi X giorni. È utile per analizzare il volume di traffico gestito dal sistema.

6. Dettaglio delle Interazioni

Il comportamento dinamico del sistema è illustrato nel **Diagramma di Sequenza** (si veda *Figura 2*), che mostra il flusso temporale dei messaggi scambiati tra i componenti interni alla rete Docker (*User Manager, Data Collector, Database*) e il mondo esterno (*Client, OpenSky Network*).

Il diagramma evidenzia sette scenari operativi principali:

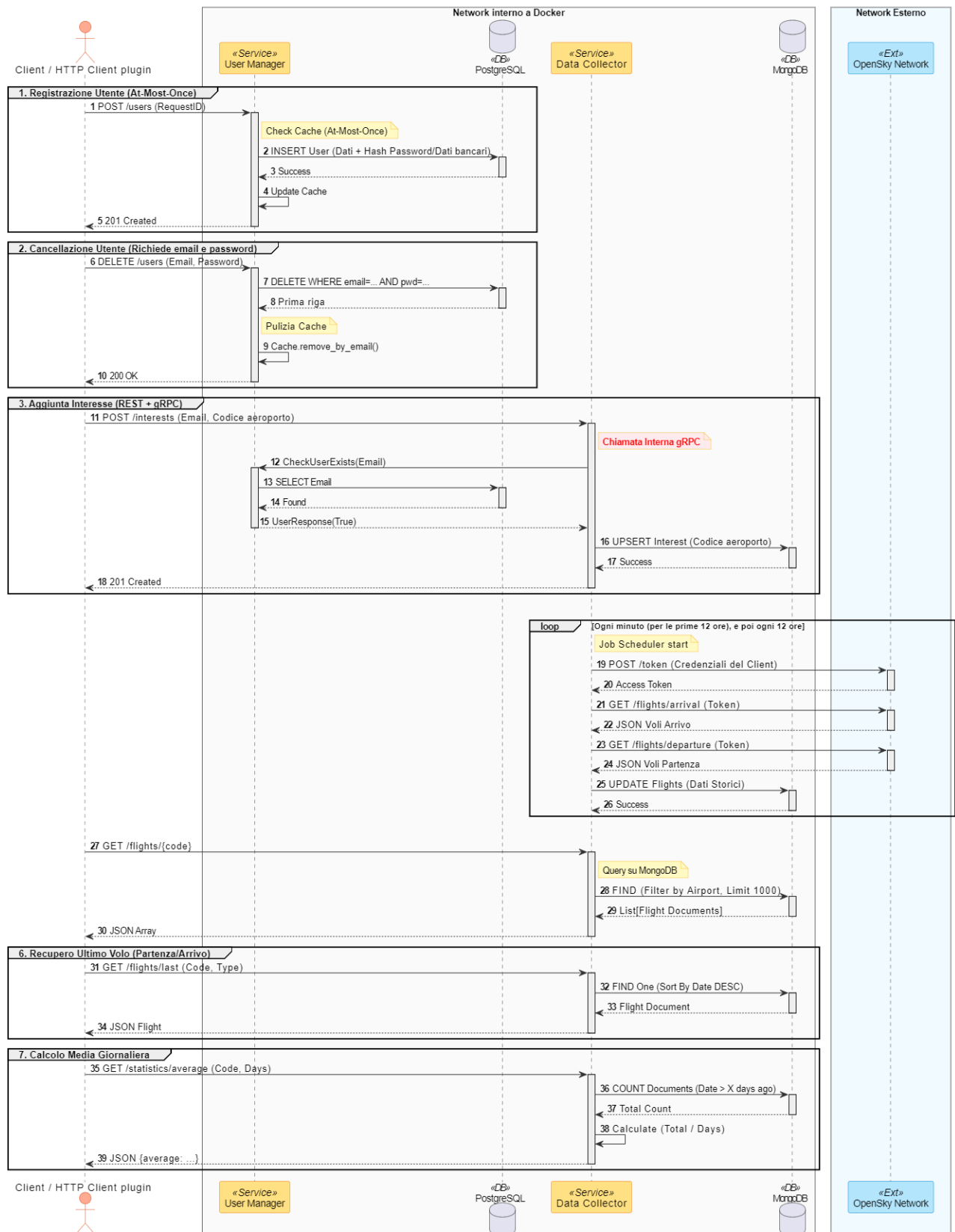


Figura 2. Diagramma che mostra nel dettaglio le interazioni, sia tra componenti dell'applicazione che tra l'applicazione il mondo esterno.

6.1 Gestione Utenti (Scenari 1 e 2)

Il primo blocco descrive il ciclo di vita dell'utente:

- **Registrazione (At-Most-Once):** Il client invia una richiesta POST includendo un *RequestID*. Lo *User Manager* verifica preliminarmente nella cache interna se l'ID è già stato processato. Solo in caso di "Cache Miss" (nuova richiesta) procede all'inserimento nel database PostgreSQL, garantendo che eventuali ritrasmissioni di rete non creino duplicati.
- **Cancellazione Sicura:** La richiesta DELETE richiede la password dell'utente. Il sistema esegue una query sul database e, in caso di successo (utente rimosso), invalida esplicitamente la cache per permettere una futura re-iscrizione con la stessa email.

6.2 Aggiunta Interessi e Comunicazione Inter-Servizio (Scenario 3)

Questo scenario illustra l'interazione ibrida tra i microservizi:

1. Il Client contatta il *Data Collector* via REST (POST /interests).
2. Prima di salvare il dato, il *Data Collector* apre un canale sincrono gRPC verso lo *User Manager*, invocando il metodo remoto `CheckUserExists`.

3. Solo dopo aver ricevuto conferma, il dato viene persistito su MongoDB tramite un'operazione di *Upsert* (operazione che combina le *insert* e *update* per gestire in modo atomico, semplice e consistente i dati).

6.3 Ciclo di Vita dello Scheduler (Loop)

Il blocco centrale del diagramma ("loop") mostra il processo asincrono gestito dallo scheduler:

- **Fase di Avvio vs Regime:** Come annotato nel diagramma, lo scheduler opera con frequenza elevata (ogni minuto) nelle prime 12 ore per popolare rapidamente il database, per poi passare a una frequenza di regime (ogni 12 ore).
- **Autenticazione OAuth2:** Per ogni ciclo, il servizio ottiene un token OAuth2 da *OpenSky*, scarica i dati di volo (sia Arrivi che Partenze) e aggiorna lo storico su MongoDB.

6.4 Interrogazione Dati e Statistiche (Scenari 5, 6, 7)

La parte finale del diagramma mostra come i dati grezzi vengano elaborati per rispondere alle richieste di lettura:

- **Lista Voli:** Recupero standard dei documenti da MongoDB filtrati per codice aeroporto.

- **Ultimo Volo:** Esecuzione di una query ottimizzata con ordinamento decrescente (Sort By Date DESC) per estrarre il singolo documento più recente.
- **Statistiche:** Calcolo aggregato in tempo reale (COUNT) sui documenti che rientrano nella finestra temporale richiesta (ultimi X giorni), restituendo al client il valore medio calcolato.

7. Esempi di Output e Validazione

In questa sezione vengono riportati i payload JSON reali restituiti dalle API durante la fase di testing, a dimostrazione del corretto funzionamento delle logiche implementate.

7.1 Verifica *At-Most-Once* (Cache Hit)

Simulando un *retry* di rete (invio dello stesso RequestID per la registrazione), il sistema riconosce la duplicazione e restituisce un codice 200 OK invece di un errore, confermando l'intervento della cache in memoria.

Per la validazione dei dati di volo e delle statistiche, abbiamo selezionato come caso di studio principale l'aeroporto di **Londra Heathrow** (codice ICAO: EGLL). Tale scelta è motivata dall'elevato volume di traffico di questo hub internazionale, che garantisce una disponibilità costante e massiva di dati sia in arrivo che in partenza, permettendo di testare efficacemente le funzionalità del sistema.

Request: POST /users (con RequestID: req-123 già usato)

Response:

```
HTTP/1.1 201 CREATED
(Headers) ...Content-Type: application/json...

{
  "email": "mario.rossi@unict.it",
  "message": "Utente creato con successo"
}

Response file saved.
> 2025-12-01T211935.201.json

Response code: 201 (CREATED); Time: 290ms (290 ms); Content length: 72 bytes (72 B)
```

7.2 Recupero Dati di Volo (OpenSky)

Esempio di un documento di volo scaricato da OpenSky e memorizzato su MongoDB, che raccoglie tutti i voli (partenza/arrivi) dell'aeroporto di Londra.

Request: GET /flights/EGLL

Response:

```
[
  {
    "_id": "692df8c3a83022b87703e92e",
    "arrivalAirportCandidatesCount": 5,
    "callsign": "BAW401 ",
    "departureAirportCandidatesCount": 2,
    "estArrivalAirport": "EGLL",
    "estArrivalAirportHorizDistance": 3189,
    "estArrivalAirportVertDistance": 66,
    "estDepartureAirport": "EGLL",
    "estDepartureAirportHorizDistance": 841,
    "estDepartureAirportVertDistance": 58,
    "fetched_at": "2025-12-01T20:21:23.670000",
    "firstSeen": 1764495173,
    "icao24": "4081bb",
    "lastSeen": 1764533995,
    "type": "arrival"
  },
  {
    "_id": "692df8c3a83022b87703e92f",
    "arrivalAirportCandidatesCount": 5,
    "callsign": "AIC2015 ",
    "departureAirportCandidatesCount": 1,
    "estArrivalAirport": "EGLL",
    "estArrivalAirportHorizDistance": 2634,
    "estArrivalAirportVertDistance": 28,
    "estDepartureAirport": "VIDP",
    "estDepartureAirportHorizDistance": 3710,
    "estDepartureAirportVertDistance": 60,
    "fetched_at": "2025-12-01T20:21:23.689000",
    "firstSeen": 1764497491,
    "icao24": "8013c2",
    "lastSeen": 1764533849,
    "type": "arrival"
  },
]
```

7.3 Statistiche Aggregata (Media Giornaliera)

Esempio della funzionalità di analisi che calcola la media dei voli in arrivo su Londra negli ultimi tre giorni.

Request: GET /statistics/average?code=EGLL&days=3&type=arrival

Response:

```
HTTP/1.1 200 OK
> (Headers) ...Content-Type: application/json...

{
  "airport": "EGLL",
  "daily_average": 28.0,
  "days_analyzed": 3,
  "total_flights": 84,
  "type": "arrival"
}

Response file saved.
> 2025-12-01T212337.200.json

Response code: 200 (OK); Time: 9ms (9 ms); Content length: 94 bytes (94 B)
```

7.4 Recupero Ultimo Volo

Dimostrazione della query specifica per ottenere il singolo volo più recente, sempre riferito all'aeroporto di Londra (ordinato per data decrescente).

Request: GET /flights/last?code=LICC&type=departure

Response:


```
{
  "_id": "692df978a83022b87703e9c0",
  "arrivalAirportCandidatesCount": 7,
  "callsign": "SWR1CG ",
  "departureAirportCandidatesCount": 3,
  "estArrivalAirport": "LSZH",
  "estArrivalAirportHorizDistance": 383,
  "estArrivalAirportVertDistance": 58,
  "estDepartureAirport": "EGLL",
  "estDepartureAirportHorizDistance": 765,
  "estDepartureAirportVertDistance": 81,
  "fetched_at": "2025-12-01T20:24:24.974000",
  "firstSeen": 1764534256,
  "icao24": "4b1800",
  "lastSeen": 1764538662,
  "type": "departure"
}
```